



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ (ИУ7)
НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.04 Программная инженерия

О т ч е т
по лабораторной работе № 2

Название: **Изучение принципов работы микропроцессорного ядра RISC-V**

Дисциплина: **Архитектура ЭВМ**

Студент гр. ИУ7-52Б

(Подпись, дата)

М. А. Жаринов
(И.О. Фамилия)

Преподаватель

(Подпись, дата)

М.А. Гейне
(И.О. Фамилия)

2025 год

Цель работы: ознакомление с принципами функционирования, построения и особенностями архитектуры суперскалярных конвейерных микропроцессоров. Дополнительной целью работы является знакомство с принципами проектирования и верификации сложных цифровых устройств с использованием языка описания аппаратуры SystemVerilog и ПЛИС.

Архитектура набора команд RV32I, являющаяся базовым подмножеством семейства RISC-V, предполагает использование 32 регистров общего назначения (x0–x31) разрядностью 32 бита и регистра счетчика команд (pc). Регистр x0 аппаратно закреплен за значением ноль, и запись в него игнорируется. Адресное пространство памяти организовано как плоское, линейное и 32-битное с адресацией байтов и порядком следования от младшего к старшему (Little Endian). Отличительной чертой архитектуры является отсутствие флагов состояния процессора; условные переходы реализуются посредством команд сравнения содержимого двух регистров. RV32I относится к архитектурам типа load-store, где арифметико-логические операции выполняются только над регистрами, а обмен данными с оперативной памятью осуществляется исключительно специальными инструкциями загрузки и сохранения.

Микроархитектура исследуемого ядра Taiga базируется на принципах конвейерной обработки с элементами суперскалярности и включает четыре основные стадии: выборку (Fetch), диспетчеризацию (ID), декодирование и планирование (Decode/Issue) и выполнение. Подсистема памяти реализована с использованием двухпортовой синхронной статической памяти ПЛИС, что обеспечивает одновременный доступ к инструкциям и данным в едином адресном пространстве с фиксированной задержкой в один такт. Такая организация памяти исключает необходимость использования кэш-памяти первого уровня. Блок декодирования совмещен с планировщиком, который распределяет инструкции между исполнительными устройствами при отсутствии конфликтов по ресурсам.

Исполнительный этап конвейера обслуживается тремя функциональными блоками: арифметико-логическим устройством (АЛУ), блоком доступа к памяти (LSU) и блоком ветвлений. АЛУ выполняет операции комбинационно за один такт, в то время как выполнение инструкций доступа к памяти занимает минимум три такта. Для предотвращения ошибок исполнения, связанных с зависимостями по данным, реализован аппаратный механизм разрешения конфликтов, приостанавливающий планирование зависимых команд. Для обработки команд передачи управления используется механизм предсказания ветвлений, позволяющий минимизировать простои конвейера; при обнаружении ошибочного предсказания производится очистка конвейера и перезапуск выборки с корректного адреса.

Задание №1

Лабораторная работа выполнялась по варианту 11. Исходный код:

```
.section .text
.globl _start;
len = 8 #Размер массива
enroll = 4 #Количество обрабатываемых элементов за одну
итерацию
elem_sz = 4 #Размер одного элемента массива

_start:
    la x1, _x
    addi x20, x1, elem_sz*len #Адрес последнего элемента
    add x31, x0, x0
lp:
    lw x2, 0(x1)
    add x31, x31, x2 #!
    lw x3, 4(x1)
    add x31, x31, x3
    lw x4, 8(x1)
    add x31, x31, x4
    lw x5, 12(x1)
    add x31, x31, x5
    addi x1, x1, elem_sz*enroll
    bne x1, x20, lp
    addi x31, x31, 1
lp2: j lp2

.section .data
_x:
    .4byte 0x1
    .4byte 0x2
    .4byte 0x3
    .4byte 0x4
    .4byte 0x5
    .4byte 0x6
    .4byte 0x7
    .4byte 0x8
```

При её сборке получен следующий дизассемблированный листинг

```
80000000 <_start>:
80000000: 00000097          auipc      x1,0x0
80000004: 04008093          addi x1,x1,64 # 80000040 <_x>
80000008: 02008a13          addi x20,x1,32
8000000c: 00000fb3          add  x31,x0,x0

80000010 <lp>:
80000010: 0000a103          lw  x2,0(x1)
80000014: 002f8fb3          add  x31,x31,x2
80000018: 0040a183          lw  x3,4(x1)
8000001c: 003f8fb3          add  x31,x31,x3
80000020: 0080a203          lw  x4,8(x1)
80000024: 004f8fb3          add  x31,x31,x4
80000028: 00c0a283          lw  x5,12(x1)
8000002c: 005f8fb3          add  x31,x31,x5
80000030: 01008093          addi x1,x1,16
80000034: fd409ee3          bne  x1,x20,80000010 <lp>
80000038: 001f8f93          addi x31,x31,1

8000003c <lp2>:
8000003c: 0000006f          jal  x0,8000003c <lp2>
```

Псевдокод на языке C, соответствующий данной программе:

```
const int len = 8;
    const int enroll = 4;
const int elem_sz = 4;
uint32_t _x[8] = {1, 2, 3, 4, 5, 6, 7, 8};

    uint32_t *x1 = _x;
    uint32_t *x20 = (char *)x1 + elem_sz*len;
uint32_t x31 = 0;
uint32_t x2, x3, x4, x5;

do
{
    x2 = x1[0];
        x31 += x2;
    x3 = x1[1];
    x31 += x3;
    x4 = x1[2];
    x31 += x4;
    x5 = x1[3];
    x31 += x5;
    x1 = (char *)x1 + elem_sz*enroll;
}
while (x1 != x20);

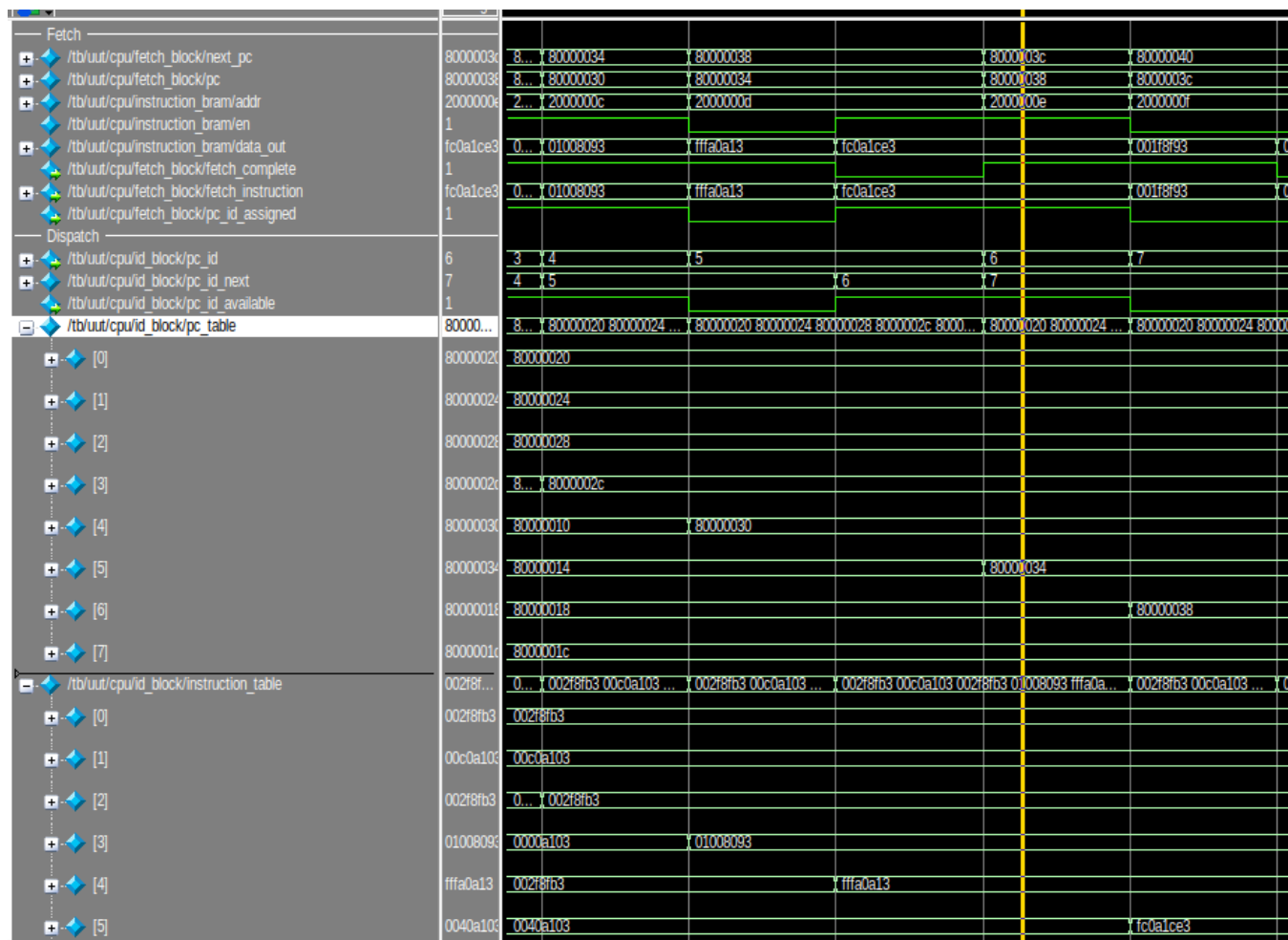
    x31 += 1;

while (1);
```

То есть программа считает сумму элементов массива в регистре x31 и прибавляет в конце 1, что означает, что в конце выполнения программы в регистре x31 будет число 37.

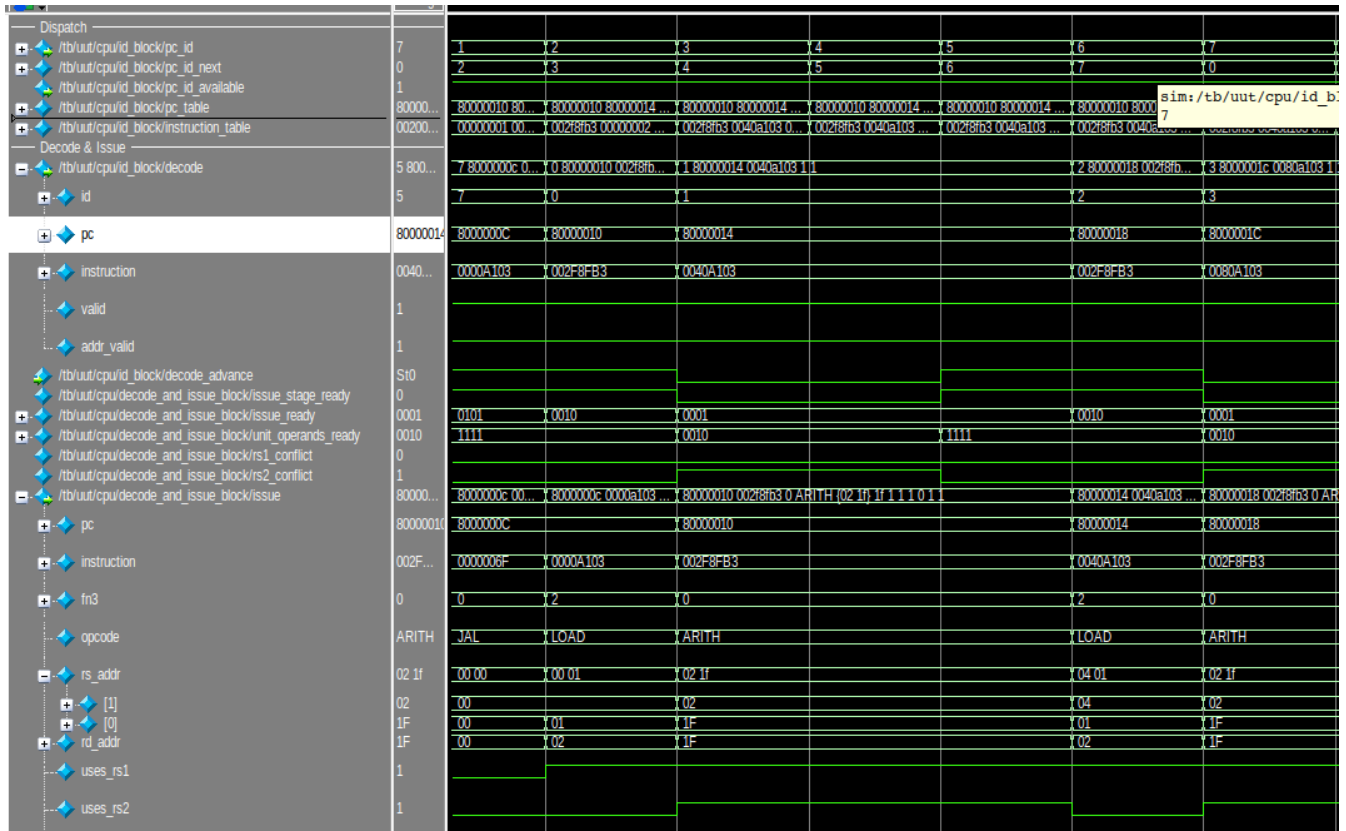
Задание №2

Получен снимок экрана, содержащий временную диаграмму выполнения стадий выборки и диспетчеризации команды с адресом 80000034 на 1-ой итерации цикла тестовой программы:



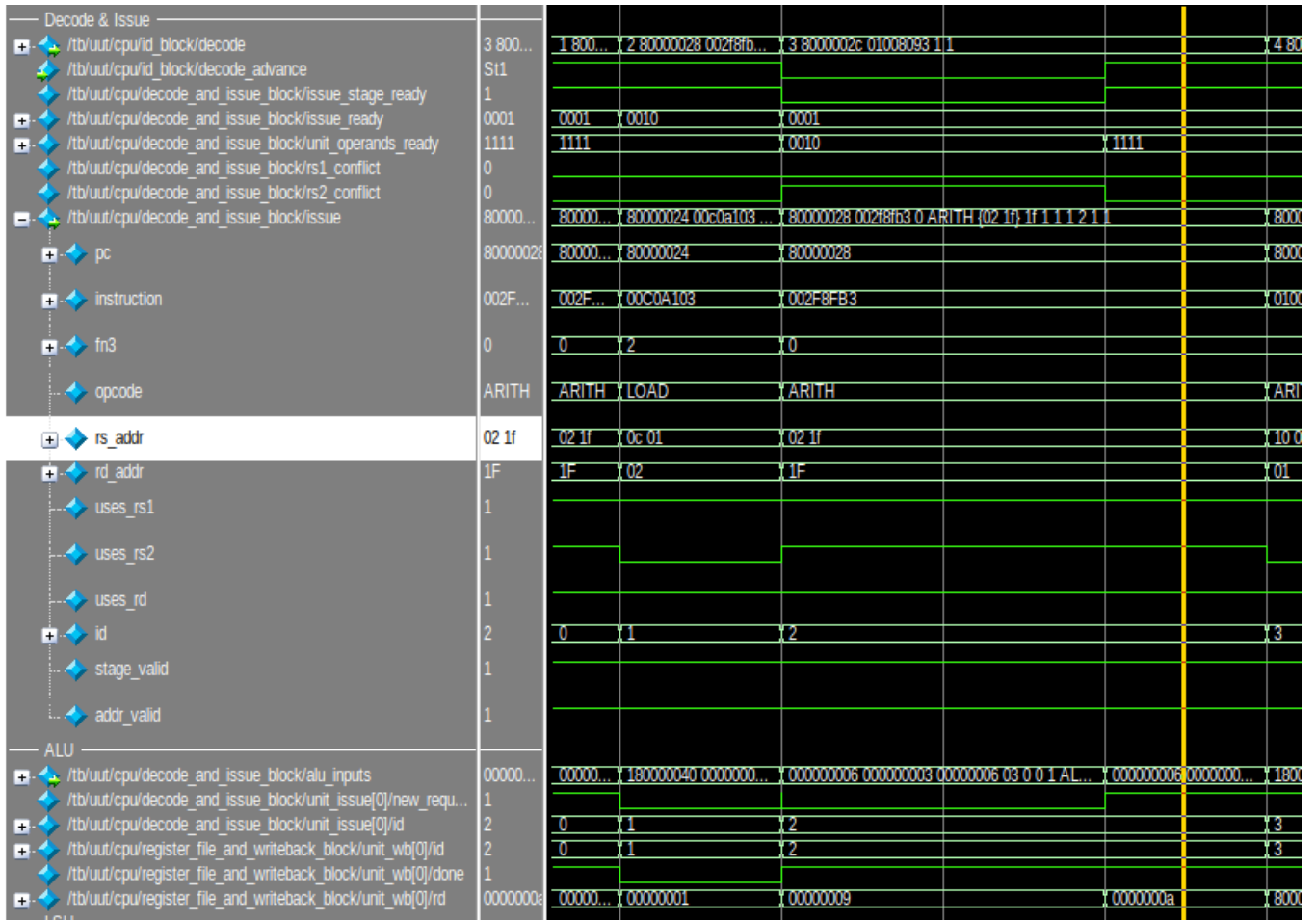
Задание №3

Получен снимок экрана, содержащий временную диаграмму выполнения стадии декодирования и планирования команды с адресом 80000014 на 2-ой итерации цикла тестовой программы:



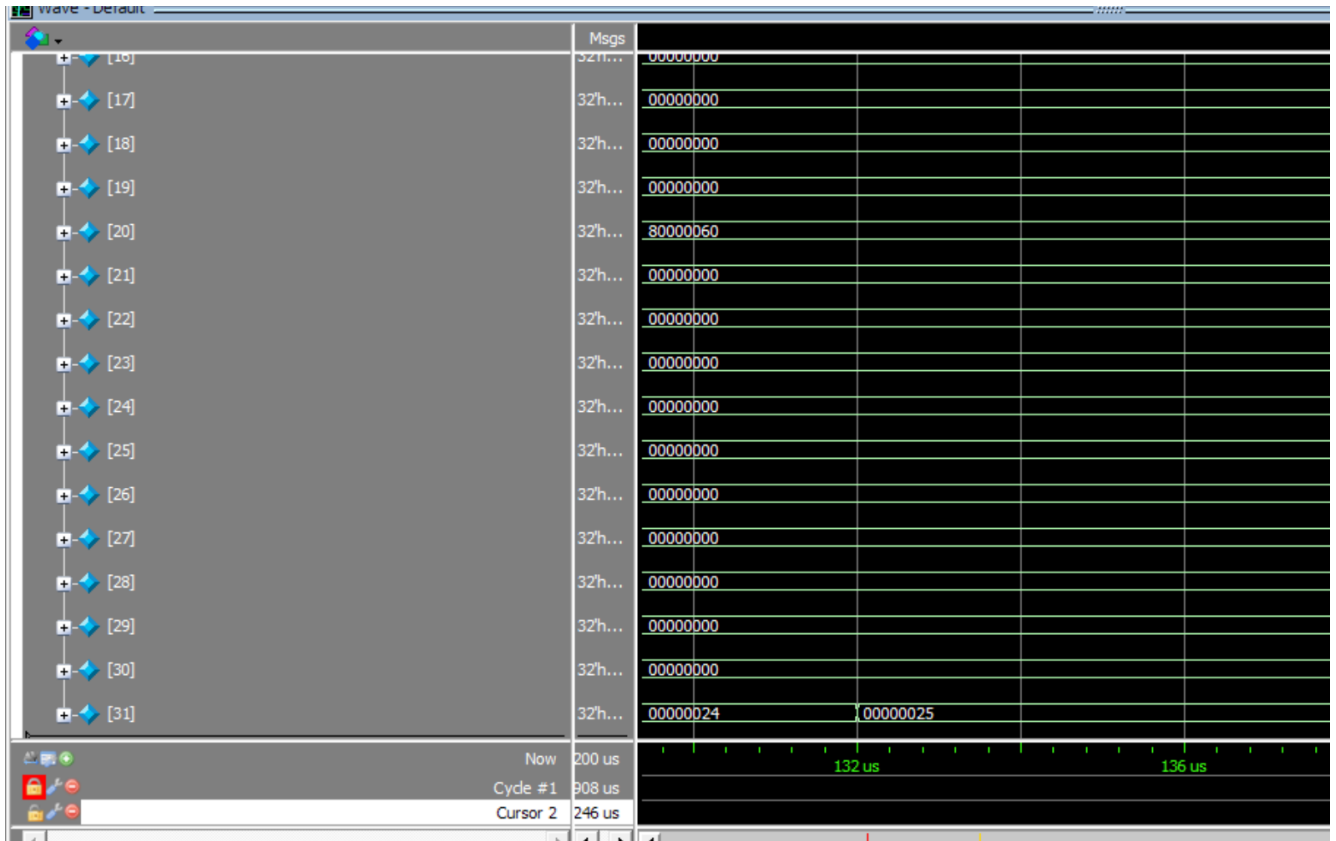
Задание №4

Получен снимок экрана, содержащий временную диаграмму выполнения стадии выполнения команды с адресом 80000028 на 1-ой итерации цикла тестовой программы:



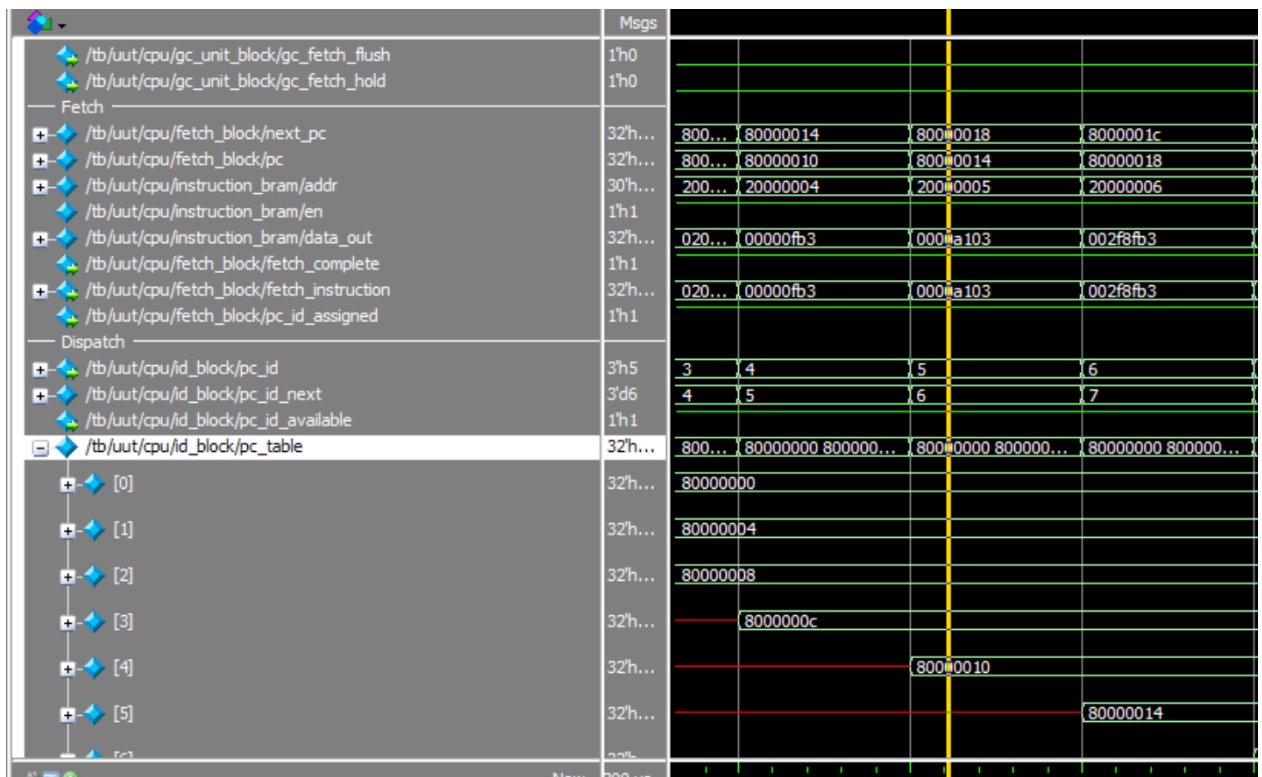
Задание №5

Была запущена симуляция программы по варианту и получено следующее значение регистра x31 в конце выполнения:

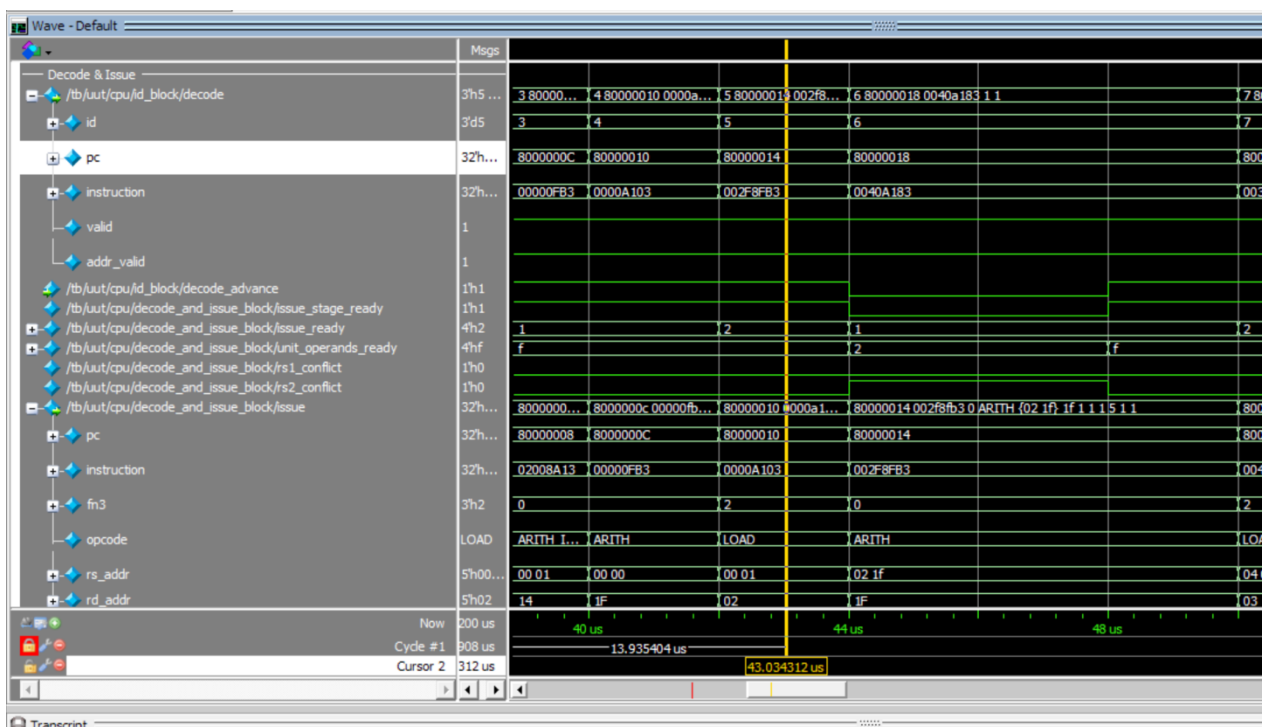


Таким образом, теоретические расчеты оказались верны, т. к. 25 в 16-ной СС = 37 в десятичной.

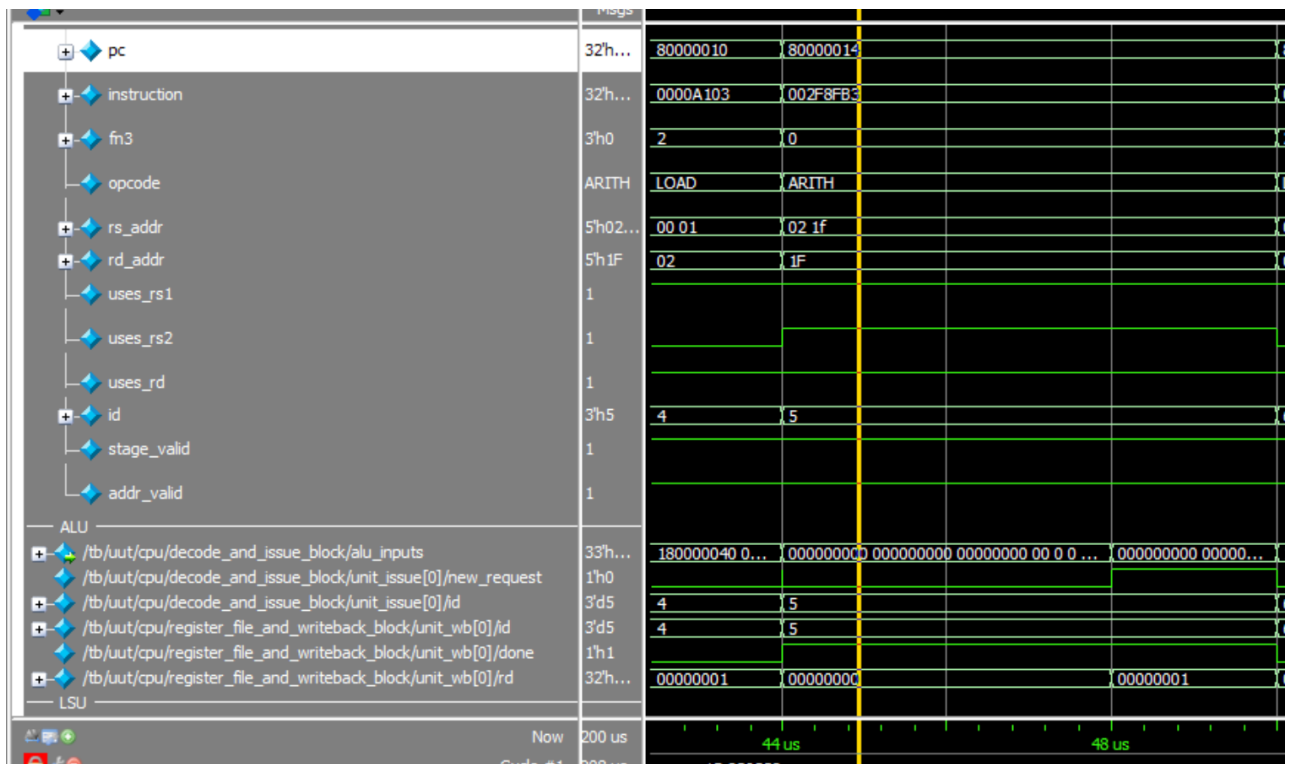
Снимок экрана, содержащий временные диаграммы сигналов, соответствующих стадии выборки и диспетчеризации команды, обозначенной в тексте программы символом #! (адрес 80000014):



Снимок экрана, содержащий временные диаграммы сигналов, соответствующих стадии декодирования и планирования этой команды:



Снимок экрана, содержащий временные диаграммы сигналов, соответствующих стадии выполнения этой команды:




```

.section .text
.globl _start;
len = 8 #Размер массива
enroll = 4 #Количество обрабатываемых элементов за одну
итерацию
elem_sz = 4 #Размер одного элемента массива

_start:
la x1, _x
addi x20, x1, elem_sz*len #Адрес последнего элемента
add x31, x0, x0

lp:
lw x2, 0(x1)
lw x3, 4(x1)
lw x4, 8(x1)
lw x5, 12(x1)
add x31, x31, x2
add x31, x31, x3
add x31, x31, x4
add x31, x31, x5
addi x1, x1, elem_sz*enroll
bne x1, x20, lp
addi x31, x31, 1
lp2: j lp2

.section .data
_x:
.4byte 0x1
.4byte 0x2
.4byte 0x3
.4byte 0x4
.4byte 0x5
.4byte 0x6
.4byte 0x7
.4byte 0x8

```

Дизассемблерный листинг оптимизированной программы:

```
80000000 <_start>:
80000000: 00000097          auipc      x1,0x0
80000004: 04008093          addi x1,x1,64 # 80000040 <_x>
80000008: 02008a13          addi x20,x1,32
8000000c: 00000fb3          add x31,x0,x0

80000010 <lp>:
80000010: 0000a103          lw x2,0(x1)
80000014: 0040a183          lw x3,4(x1)
80000018: 0080a203          lw x4,8(x1)
8000001c: 00c0a283          lw x5,12(x1)
80000020: 002f8fb3          add x31,x31,x2
80000024: 003f8fb3          add x31,x31,x3
80000028: 004f8fb3          add x31,x31,x4
8000002c: 005f8fb3          add x31,x31,x5
80000030: 01008093          addi x1,x1,16
80000034: fd409ee3          bne x1,x20,80000010 <lp>
80000038: 001f8f93          addi x31,x31,1

8000003c <lp2>:
8000003c: 0000006f          jal x0,8000003c <lp2>
```

Трасса выполнения оптимизированной программы:

[illegible]

Таким образом, оптимизированная программа выполняется за 42 такта, в то время как исходная за 58 тактов, то есть оптимизация ускорила выполнение на 16 тактов или на 28%.

Вывод

В процессе выполнения лабораторной работы я ознакомился с принципами функционирования, построения и особенностями архитектуры суперскалярных конвейерных микропроцессоров, познакомился с принципами проектирования и верификации сложных цифровых устройств. Оптимизация программы позволила ускорить ее выполнение на 28%.